3. Git-HOL

Git Hands-on Lab 3 Report – Branching and Merging

To create a new Git branch, make changes in the branch, merge the changes into the master branch, view the commit history, and delete the merged branch.

Prerequisites

- Git installed and configured.
- Existing **GitLab** repository.
- Notepad++ configured as the default editor.
- P4Merge configured (optional for visual comparison).

Step 1: Open the Repository

Open Git Bash and navigate to the existing GitLab repository.

cd ~/GitLab

pwd

Step 2: Create a New Branch

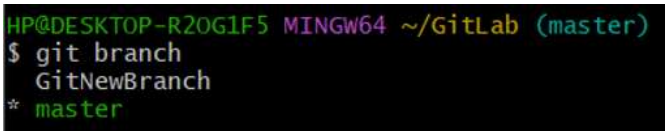
Create a new branch named **GitNewBranch**.

git branch GitNewBranch

Step 3: List Available Branches

Display all the available branches and verify the current branch.

git branch



Step 4: Switch to the New Branch

Switch from the **master** branch to **GitNewBranch**.

git checkout GitNewBranch

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ git checkout GitNewBranch
Switched to branch 'GitNewBranch'

HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (GitNewBranch)
$ git branch
* GitNewBranch
  master

HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (GitNewBranch)
$
```

Step 5: Create a New File

Create a new file named **branch.txt** with sample content.

echo "This file belongs to GitNewBranch." > branch.txt

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (GitNewBranch)
$ echo "This file belongs to GitNewBranch." > branch.txt
```

Step 6: Check Repository Status

Verify the repository status before staging the file.

git status

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (GitNewBranch)
$ git status
On branch GitNewBranch
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        branch.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Step 7: Stage the File

Add the newly created file to the staging area.

git add branch.txt

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (GitNewBranch)
$ git add branch.txt
warning: in the working copy of 'branch.txt', LF will be replaced by CRLF the next time Git touches it
```

Step 8: Commit the Changes

Commit the staged changes in **GitNewBranch**

git commit -m "Added branch.txt in GitNewBranch"

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (GitNewBranch)
$ git commit -m "Added branch.txt in GitNewBranch"
[GitNewBranch caaa663] Added branch.txt in GitNewBranch
 1 file changed, 1 insertion(+)
 create mode 100644 branch.txt
```

Step 9: Verify Repository Status

Check the repository status after committing the changes.

git status

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (GitNewBranch)
$ git status
On branch GitNewBranch
nothing to commit, working tree clean
```

Step 10: Switch Back to Master Branch

Switch back to the master branch.

git checkout master

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (GitNewBranch)
$ git checkout master
Switched to branch 'master'
```

Step 11: Compare the Branches

Compare the differences between the master branch and GitNewBranch.

git diff master GitNewBranch

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ git diff master GitNewBranch
diff --git a/branch.txt b/branch.txt
new file mode 100644
index 0000000..3752399
--- /dev/null
+++ b/branch.txt
@@ -0,0 +1 @@
+This file belongs to GitNewBranch.
```

Step 12: Merge the Branch

Merge **GitNewBranch** into the master branch.

git merge GitNewBranch

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ git merge GitNewBranch
Updating a3748af..caaa663
Fast-forward
 branch.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 branch.txt
```

Step 13: View Commit History

Display the commit history in graphical format.

git log --oneline --graph --decorate

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ git log --oneline --graph --decorate
* caaa663 (HEAD -> master, GitNewBranch) Added branch.txt in GitNewBranch
* a3748af Added .gitignore
* 15f4d2e Added hello.txt
```

Step 14: Delete the Merged Branch

Delete **GitNewBranch** after merging.

git branch -d GitNewBranch

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ git branch -d GitNewBranch
Deleted branch GitNewBranch (was caaa663).
```

Step 15: Final Verification

Verify that only the master branch exists and the repository is clean.

git branch

git status

```
HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ git branch
* master

HP@DESKTOP-R2OG1F5 MINGW64 ~/GitLab (master)
$ git status
On branch master
nothing to commit, working tree clean
```

Result

The branch **GitNewBranch** was created successfully, and a new file was added and committed. The branch was merged into the **master** branch successfully, the commit history was verified, and the repository remained in a clean working state with the message **"nothing to commit, working tree clean**